

Departement Informatik
Hochschule Luzern (Schweiz)

Faire Wohnungsvergabe Tool

WEB PROGRAMMING LAB PROJEKT

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

von

Noah Emmenegger

von

Zug (Schweiz)

Web Programming Lab Projekt vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

Titel: Faire Wohnungsvergabe Tool

Student: Noah Emmenegger

Studiengang: BSc Informatik

Jahr: 2026

Betreuungsperson: Dozent/in

Auftraggeberin / Auftraggeber: Web Programming Lab

Kurzfassung

[Kurzfassung des Projekts]

Inhaltsverzeichnis

1	Einführung und Ziele	10
1.1	Aufgabenstellung	10
1.1.1	Fachlicher Umfang	10
1.1.2	Abgrenzung	12
1.2	Qualitätsziele	12
1.3	Stakeholder	12
2	Randbedingungen	14
2.1	Technische Randbedingungen	14
2.2	Organisatorische Randbedingungen	14
2.3	Konventionen	14
3	Kontextabgrenzung	15
3.1	Fachlicher Kontext	15
3.2	Technischer Kontext	15
4	Lösungsstrategie	17
4.1	Technologie-Stack	17
4.2	Architekturansatz	18
4.3	Entwicklungsvorgehen	18
5	Bausteinsicht	19
5.1	Ebene 1: Gesamtsystem	19
5.2	Ebene 2: Frontend	19
5.3	Ebene 2: Backend	19
6	Laufzeitsicht	20
6.1	Ressource erstellen oder ändern	20
6.2	Ressource löschen	20
6.3	Zweite Darstellungsform laden	20
7	Verteilungssicht	21
7.1	Produktionsumgebung	21
7.2	Reproduzierbarer Start	21
8	Querschnittliche Konzepte	22
8.1	Domänenmodell und Persistenz	22
8.2	API- und Schnittstellendesign	22
8.3	Benutzeroberfläche und Responsive Design	22
8.4	Fehlerbehandlung und Logging	22
8.5	Sicherheit	22
8.6	Testkonzept	22
9	Architekturentscheidungen	23
9.1	ADR-001: [Entscheidungstitel]	23
10	Qualitätsanforderungen	24
10.1	Qualitätsbaum	24
10.2	Qualitätsszenarien	24
10.3	Evaluationsergebnisse	24
11	Risiken und technische Schulden	25
12	Glossar	26

13	Fazit und Reflexion	27
13.1	Was ist gut gelaufen?	27
13.2	Wo lagen die Herausforderungen?	27
13.3	Was würde ich beim nächsten Mal anders machen?	27
13.4	Zielerreichung	27
14	Arbeitsjournal	28
15	Anhang	29
15.1	Abgabe und Softwareartefakte	29
15.2	Ergänzende Unterlagen	29

Abkürzungsverzeichnis

Abkürzung	Bedeutung
CMS	Content Management System
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets

Literaturverzeichnis

Abbildungsverzeichnis

Abbildung 1	Fachlicher Kontext der Wohnungsvergabe-Plattform	15
Abbildung 2	Bausteinsicht des Gesamtsystems	19
Abbildung 3	Verteilung der Anwendung in Produktion	21

Tabellenverzeichnis

Code-Verzeichnis

1 Einführung und Ziele

Dieses Kapitel beschreibt den fachlichen Anlass, den Umfang, die wichtigsten Qualitätsziele und die beteiligten Personen des Projekts.

1.1 Aufgabenstellung

Immobilien- und Genossenschaftswirtschaften möchten Wohnungsvergaben möglichst fair und nachvollziehbar durchführen. Heute müssen Ausschreibungen, Bewerbungsformulare und eingehende Bewerbungen häufig mit mehreren voneinander getrennten Hilfsmitteln verwaltet und manuell verglichen werden.

Im Rahmen des Projekts wird deshalb eine webbasierte Plattform für die Vorbereitung und Durchführung von Wohnungsvergaben entwickelt. Mitarbeitende einer Gesellschaft erstellen damit Wohnungsausschreibungen und individuelle Bewerbungsformulare. Vor Beginn einer Ausschreibung definieren sie Bewertungskriterien und deren Gewichtung. Eingehende Bewerbungen werden anschliessend zentral verwaltet, anhand dieser Regeln automatisch bewertet und nach ihrer Übereinstimmung mit den Vergabekriterien sortiert.

Die Plattform unterstützt die Entscheidungsfindung, trifft aber keine endgültige Vergabeentscheidung. Diese bleibt bei der zuständigen Person. Damit die Entscheidung überprüfbar bleibt, zeigt das System für jede Bewerbung transparent auf, wie die Gesamtbewertung zustande gekommen ist.

1.1.1 Fachlicher Umfang

Die Anforderungen sind nach der MoSCoW-Methode priorisiert. Die Must-Have-Stories bilden den geplanten Mindestumfang der ersten Version.

Must Have

- Als Gesellschaft kann ich eine neue Wohnungsausschreibung erstellen.
- Als Gesellschaft kann ich für eine Ausschreibung ein individuelles Bewerbungsformular erstellen.
- Als Gesellschaft kann ich Bewertungskriterien für eine Ausschreibung definieren.
- Als Gesellschaft kann ich festlegen, wie stark die einzelnen Kriterien gewichtet werden.
- Als Gesellschaft kann ich eine Musterbewerbung ausfüllen, um die definierten Kriterien und Gewichtungen zu testen.

- Als Gesellschaft kann ich die Kriterien und Gewichtungen vor Beginn der Ausschreibung festlegen und sperren.
- Als Gesellschaft kann ich das Bewerbungsformular über einen Link teilen.
- Als Gesellschaft kann ich das Bewerbungsformular optional mit einem Zugangsschlüssel schützen.
- Als Bewerber/in kann ich das Bewerbungsformular ausfüllen und absenden.
- Als Gesellschaft kann ich alle eingegangenen Bewerbungen zentral einsehen.
- Als Gesellschaft kann ich die Bewerbungen anhand der definierten Kriterien automatisch bewerten lassen.
- Als Gesellschaft kann ich die Bewerbungen nach ihrer Übereinstimmung mit den Vergabekriterien sortieren.
- Als Gesellschaft kann ich bei einer Bewerbung einsehen, wie sich die einzelnen Kriterien auf die Gesamtbewertung ausgewirkt haben.
- Als Gesellschaft kann ich eine Bewerbung als ausgewählt markieren.
- Als Gesellschaft kann ich Bewerbungen und deren Bewertungen exportieren.

Should Have

- Als Gesellschaft kann ich bestehende Ausschreibungen duplizieren und als Vorlage verwenden.
- Als Gesellschaft kann ich bestehende Bewerbungsformulare wiederverwenden.
- Als Gesellschaft kann ich Mindestanforderungen definieren, die eine Bewerbung erfüllen muss.
- Als Gesellschaft kann ich Bewerbungen filtern und durchsuchen.
- Als Gesellschaft kann ich mehrere Mitarbeitende verwalten.
- Als Gesellschaft kann ich abgeschlossene Ausschreibungen archivieren.
- Als Gesellschaft kann ich einen zusammenfassenden Bericht über eine abgeschlossene Vergabe erstellen.

Could Have

- Als Bewerber/in kann ich den aktuellen Status meiner Bewerbung einsehen.
- Als Gesellschaft kann ich Bewerber/innen direkt über den Status ihrer Bewerbung informieren.
- Als Gesellschaft kann ich Statistiken zu vergangenen Ausschreibungen anzeigen.

- Als Gesellschaft kann ich unterschiedliche Bewertungsmodelle als Vorlagen speichern.
- Als Gesellschaft kann ich einzelne Bewerbungen kommentieren oder intern markieren.

1.1.2 Abgrenzung

Folgende Funktionen sind für den Projektzeitraum ausdrücklich nicht vorgesehen:

- Die Plattform trifft keine endgültige Vergabeentscheidung ohne menschliche Bestätigung.
- Es wird keine künstliche Intelligenz zur Bewertung oder Vorhersage der Eignung von Bewerber/innen eingesetzt.
- Es wird kein ausschreibungsübergreifender Score für Bewerber/innen erstellt.
- Es werden keine externen Datenquellen oder sozialen Netzwerke zur Bewertung verwendet.

1.2 Qualitätsziele

Prio	Qualitätsziel	Begründung	Nachweis
1	Nachvollziehbarkeit	Eine Bewertung muss aus den vorab festgelegten Kriterien, Antworten und Gewichtungen vollständig erklärbar sein.	Berechnungsbeispiele, Unit- und E2E-Tests
2	Benutzbarkeit	Die Kernaufgaben sind auf Desktop und Mobile verständlich und effizient ausführbar.	E2E-Test, manuelle Prüfung, Lighthouse
3	Zuverlässigkeit	Ausschreibungen, Kriterien und Bewerbungen bleiben konsistent und dauerhaft verfügbar.	Integrations- und E2E-Tests
4	Änderbarkeit	Die Anwendung bleibt trotz des begrenzten Projektumfangs nachvollziehbar strukturiert und erweiterbar.	Code-Review, automatisierte Tests

1.3 Stakeholder

Stakeholder	Erwartung	Bezug zum System
Mitarbeitende einer Immobiliengesellschaft	Einfache Verwaltung von Ausschreibungen und Bewerbungen	Hauptbenutzer/innen des geschützten Verwaltungsbereichs

Stakeholder	Erwartung	Bezug zum System
schaft oder Genossenschaft	sowie nachvollziehbare Entscheidungsgrundlagen	
Bewerber/innen	Ein verständliches, responsives und zuverlässig funktionierendes Bewerbungsformular	Füllen ein öffentlich geteiltes und optional geschütztes Formular aus
Verantwortliche der Gesellschaft	Ein fairer, dokumentierbarer Vergabeprozess; die endgültige Entscheidung bleibt in ihrer Verantwortung	Verantworten Kriterien und Vergabeentscheid
Entwickler/in und Betreiber/in	Verständliche Struktur, testbare Fachlogik und reproduzierbarer Betrieb	Entwickelt, wartet und betreibt das System
Dozent/in	Nachvollziehbare Erfüllung der Projektanforderungen	Bewertet Artefakt, Vorgehen und Dokumentation

2 Randbedingungen

Dieses Kapitel enthält Vorgaben, die den Lösungsraum des Projekts einschränken. Gewählte Technologien und Architekturansätze werden dagegen in Kapitel 4 beschrieben.

2.1 Technische Randbedingungen

- Die Anwendung muss Daten persistent in einer Datenbank speichern.
- Die Benutzeroberfläche muss auf Desktop, Tablet und Mobile responsiv nutzbar sein.
- Automatisierte Unit-, Integrations- und/oder E2E-Tests müssen passend zu den Projektrisiken eingesetzt werden.
- Ein Produktions-Build muss über eine öffentliche URL oder mit `docker compose up` reproduzierbar startbar sein.
- Der durchschnittliche Lighthouse-Score muss für Mobile und Desktop mindestens 90 betragen.
- Sämtliche Artefakte werden in einem Git-Repository abgegeben.

2.2 Organisatorische Randbedingungen

- Das Projekt wird als Einzelarbeit mit einem vorgesehenen persönlichen Arbeitsaufwand von rund 60 Stunden durchgeführt.
- Die Entwicklung erfolgt strukturiert und systematisch.
- Wesentliche Architekturentscheidungen müssen nachvollziehbar begründet werden.
- Die Dokumentation richtet sich an Software Engineers und bleibt kurz, präzise und diagrammorientiert.

2.3 Konventionen

- Quellcode und Dokumentation werden gemeinsam mit Git versioniert.
- Die Architekturdokumentation orientiert sich an der Kapitelstruktur von arc42.
- Fachbegriffe werden im gesamten System und in der Dokumentation einheitlich verwendet; projektspezifische Begriffe werden im Glossar erläutert.
- Eine User Story gilt erst als abgeschlossen, wenn sie umgesetzt, angemessen getestet und in der Dokumentation beziehungsweise im Arbeitsjournal berücksichtigt ist.

3 Kontextabgrenzung

Dieses Kapitel grenzt die Wohnungsvergabe-Plattform von ihren Benutzergruppen und der technischen Umgebung ab. In der ersten Version bestehen keine Schnittstellen zu externen Fachsystemen oder Datenquellen.

3.1 Fachlicher Kontext

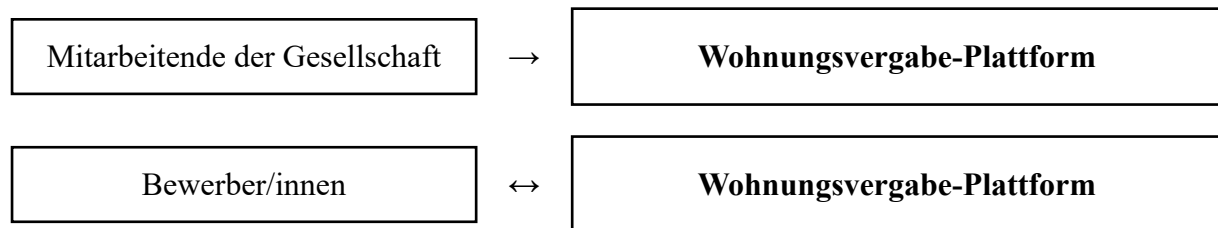


Abbildung 1: Fachlicher Kontext der Wohnungsvergabe-Plattform

Akteur	Schnittstelle	Ausgetauschte Informationen
Mitarbeitende der Gesellschaft	Geschützte Weboberfläche	Ausschreibungen, Formulare, Kriterien, Gewichtungen, Musterbewerbungen, Bewerbungen, Auswahlstatus und Exporte
Bewerber/innen	Öffentlich geteiltes Webformular, optional mit Zugangsschlüssel	Formularfelder, Bewerbungsdaten und - je nach umgesetztem Umfang - Bewerbungsstatus

Die Gesellschaft legt die Vergaberegeln fest und erhält die aufbereiteten Bewertungen. Bewerber/innen übermitteln ausschliesslich die im jeweiligen Formular verlangten Angaben. Die endgültige Vergabeentscheidung entsteht ausserhalb des Systems durch die verantwortliche Person; im System wird sie lediglich dokumentiert.

3.2 Technischer Kontext

Die Benutzer/innen greifen mit einem aktuellen Webbrowser über HTTPS auf die Anwendung zu. Frontend und Backend werden gemeinsam als Next.js-Anwendung betrieben. Das Backend persistiert die Daten über ein ORM in PostgreSQL. Datenexporte werden als Datei über die Weboberfläche bereitgestellt.

Kommunikations-partner	Kanal/Protokoll	Bemerkung
Browser der Mitarbeitenden	HTTPS; HTML, CSS und JavaScript	Zugriff auf den authentisierten Verwaltungsbereich
Browser der Bewerber/innen	HTTPS; HTML, CSS und JavaScript	Zugriff über einen Freigabelink; optional durch einen Zugangsschlüssel geschützt
PostgreSQL-Datenbank	Datenbankprotokoll über ORM	Persistenz für Ausschreibungen, Formulare, Kriterien, Bewerbungen und Bewertungen
Dateisystem des Browsers	HTTPS-Download	Export der Bewerbungen und ihrer Bewertungen

Externe Datenquellen, soziale Netzwerke und KI-Dienste liegen ausserhalb der Systemgrenze und werden nicht angebunden.

4 Lösungsstrategie

Dieses Kapitel fasst die grundlegenden Entscheidungen zusammen, mit denen die fachlichen Ziele und Qualitätsziele erreicht werden. Detaillierte und später revidierbare Entscheidungen werden in Kapitel 9 als Architecture Decision Records festgehalten.

4.1 Technologie-Stack

Bereich	Technologie	Begründung und Zielbeitrag
Web-Framework	Next.js mit TypeScript	Frontend und Backend können in einer Anwendung entwickelt und als gemeinsames Artefakt ausgeliefert werden. TypeScript unterstützt die konsistente Modellierung der fachlichen Daten.
Benutzeroberfläche	React und Tailwind CSS	Komponentenbasierte Umsetzung der Verwaltungsoberfläche und der dynamischen, responsiven Bewerbungsformulare.
Backend	Next.js auf Node.js	Serverseitige Validierung, Fachlogik und Datenzugriff bleiben innerhalb des Monolithen gebündelt.
Datenbank	PostgreSQL	Relationale und transaktionale Speicherung der miteinander verknüpften Ausschreibungen, Kriterien und Bewerbungen.
Datenzugriff	Prisma oder Drizzle ORM; Entscheid offen	Typsicherer Datenzugriff und nachvollziehbare Schema-Migrationen; die konkrete Bibliothek wird in einem ADR entschieden.
Authentisierung	Einfache Benutzeranmeldung für Gesellschaften	Schützt den Verwaltungsbereich; öffentlich geteilte Formulare können zusätzlich einen Zugangsschlüssel verlangen.
Tests	Unit-, Integrations- und E2E-Tests; Werkzeuge noch offen	Die deterministische Bewertungslogik, Datenintegrität und zentralen Benutzerabläufe werden automatisiert abgesichert.
Betrieb	Docker	Ermöglicht einen reproduzierbaren Produktionsbetrieb und erfüllt die Abgabeanforderungen.

Bereich	Technologie	Begründung und Zielbeitrag
Versionsverwaltung	Git und GitHub	Nachvollziehbare Entwicklung und gemeinsame Ablage aller Artefakte.

4.2 Architekturansatz

Die Anwendung wird als modular strukturierter Monolith umgesetzt. Frontend, Backend und Fachlogik befinden sich in derselben Next.js-Anwendung und werden gemeinsam ausgeliefert. Innerhalb des Monolithen werden die fachlichen Verantwortlichkeiten - insbesondere Ausschreibungen, Formulare, Kriterien und Bewertung, Bewerbungen, Benutzerverwaltung und Export - klar voneinander getrennt. Dadurch bleibt der Betrieb einfach, ohne auf eine nachvollziehbare und erweiterbare Struktur zu verzichten.

Die Bewertung einer Bewerbung erfolgt regelbasiert und deterministisch. Antworten werden mit den vor Beginn einer Ausschreibung festgelegten Kriterien verglichen und anhand der gespeicherten Gewichtungen zu einer Gesamtbewertung zusammengeführt. Kriterien und Gewichtungen werden mit dem Start der Ausschreibung gesperrt. Neben dem Gesamtwert speichert beziehungsweise liefert die Anwendung die Beiträge der einzelnen Kriterien, damit das Ranking jederzeit erklärt und mit Berechnungsbeispielen getestet werden kann.

PostgreSQL stellt die konsistente Persistenz der fachlichen Daten sicher. Schreiboperationen werden serverseitig validiert und für zusammengehörige Änderungen transaktional ausgeführt. Responsive Komponenten, automatisierte Tests der risikoreichen Abläufe und regelmäßige Lighthouse-Messungen unterstützen die Qualitätsziele Benutzbarkeit, Zuverlässigkeit und Änderbarkeit.

4.3 Entwicklungsvorgehen

Die Entwicklung erfolgt iterativ entlang der priorisierten User Stories. Zuerst wird ein durchgängiger Kernablauf von der Ausschreibung bis zur bewerteten Bewerbung umgesetzt; weitere Must-, Should- und Could-Have-Stories werden anschliessend in dieser Reihenfolge ergänzt. Jede Iteration umfasst Implementierung, angemessene automatisierte Tests, eine kurze Überprüfung der Qualitätsziele und einen Eintrag im Arbeitsjournal. Technisch riskante oder noch offene Entscheide werden früh mit kleinen Versuchen geprüft und als ADR dokumentiert.

5 Bausteinsicht

5.1 Ebene 1: Gesamtsystem

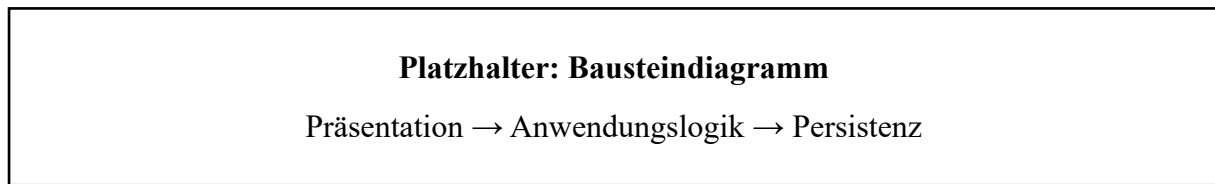


Abbildung 2: Bausteinsicht des Gesamtsystems

Baustein	Verantwortung	Schnittstellen
[Frontend]	[Darstellung und Benutzerinteraktion]	[HTTP/API]
[Backend]	[Fachlogik und Datenzugriff]	[REST/GraphQL/...]
[Datenbank]	[Persistente Datenhaltung]	[DB-Protokoll/ORM]

5.2 Ebene 2: Frontend

[Komponenten, Verantwortlichkeiten, Abhängigkeiten und bewusste Abweichungen]

5.3 Ebene 2: Backend

[Module/Schichten, Verantwortlichkeiten, Abhängigkeiten und bewusste Abweichungen]

6 Laufzeitsicht

6.1 Ressource erstellen oder ändern

1. Anwender/in sendet gültige Eingaben ab.
2. Frontend prüft die Eingaben und sendet die Anfrage an das Backend.
3. Backend validiert die Daten fachlich und speichert sie in der Datenbank.
4. Frontend aktualisiert die Darstellung oder zeigt einen verständlichen Fehler.

6.2 Ressource löschen

[Ablauf inklusive Bestätigung, Autorisierung, Fehlerbehandlung und UI-Aktualisierung]

6.3 Zweite Darstellungsform laden

[Ablauf der Datenaufbereitung für die zweite inhaltlich unterschiedliche Sicht]

7 Verteilungssicht

7.1 Produktionsumgebung

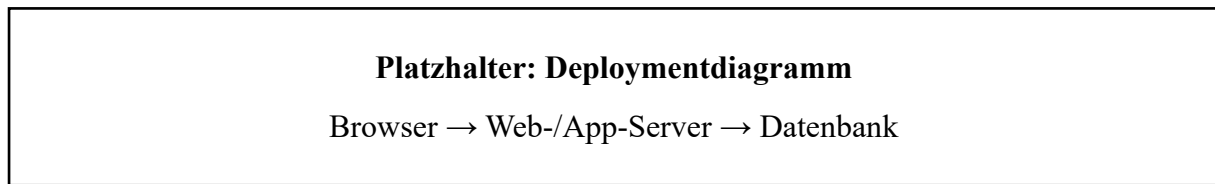


Abbildung 3: Verteilung der Anwendung in Produktion

Knoten	Artefakt	Konfiguration/Verantwortung
Client	Browser-Anwendung	Auslieferung und unterstützte Browser
[Server/Plattform]	[Produktions-Bundle]	[Start, Umgebungsvariablen, Ska- lierung]
[Datenbank]	[Schema und Daten]	[Migration, Persistenz, Backup]

7.2 Reproduzierbarer Start

`docker compose up`

[Falls öffentlich deployed: URL und Deployment-Ablauf]

8 Querschnittliche Konzepte

8.1 Domänenmodell und Persistenz

[Domänen-/Datenmodell mit kurzer Erklärung]

8.2 API- und Schnittstellendesign

[Routen/Operationen, Datenformate, Validierung, Versionierung und Fehlerantworten]

8.3 Benutzeroberfläche und Responsive Design

[Layoutprinzipien, Breakpoints, Barrierefreiheit und die zwei Darstellungsformen]

8.4 Fehlerbehandlung und Logging

[Fehlerkategorien, Meldungen für Benutzer/innen, technische Logs und Datenschutz]

8.5 Sicherheit

[Eingabevalidierung, Geheimnisse, Abhängigkeiten, Authentisierung/Autorisierung – soweit relevant]

8.6 Testkonzept

Teststufe	Zweck	Beispiele/Werkzeuge
Unit	Fachlogik isoliert prüfen	[Werkzeug und zentrale Fälle]
Integration	Datenbank und Schnittstellen prüfen	[Werkzeug und zentrale Fälle]
E2E	Kritische User Journeys prüfen	[Werkzeug und zentrale Fälle]
Qualität	Performance, Accessibility und Best Practices	Lighthouse Mobile und Desktop

9 Architekturentscheidungen

Entscheidungen werden als kurze Architecture Decision Records (ADR) festgehalten. Eine Entscheidung beschreibt Kontext und Folgen, nicht nur das gewählte Werkzeug.

9.1 ADR-001: [Entscheidungstitel]

Status	Vorgeschlagen / Akzeptiert / Ersetzt
Kontext	[Problem, Randbedingungen und Entscheidungstreiber]
Betrachtete Optionen	[Option A, Option B, Option C]
Entscheidung	[Gewählte Option und nachvollziehbare Begründung]
Konsequenzen	[Positive und negative Folgen, Risiken und Folgemaßnahmen]

10 Qualitätsanforderungen

10.1 Qualitätsbaum

- Qualität
 - Nachvollziehbarkeit: deterministische Bewertung, erklärbare Teilbewertungen
 - Benutzbarkeit: responsive Bedienung, verständliche Rückmeldungen
 - Änderbarkeit: klare Verantwortlichkeiten, automatisierte Tests
 - Zuverlässigkeit: Datenintegrität, kontrollierte Fehlerbehandlung
 - Effizienz: Lighthouse-Durchschnitt mindestens 90 auf Mobile und Desktop

10.2 Qualitätsszenarien

ID	Merkmal	Szenario	Messgrösse	Nachweis
QS-01	Performance	Eine typische Seite wird in der Produktionsversion aufgerufen.	Lighthouse-Durchschnitt ≥ 90 , Mobile und Desktop	Lighthouse-Bericht
QS-02	Benutzbarkeit	Eine Kernfunktion wird auf einem kleinen Bildschirm ausgeführt.	Ohne horizontales Scrollen; alle Aktionen erreichbar	E2E/Manuell
QS-03	Zuverlässigkeit	Ungültige Daten werden übermittelt.	Keine inkonsistenten Daten; verständliche Fehlermeldung	Integration/E2E
QS-04	Änderbarkeit	Ein neues Bewertungskriterium oder ein neuer Formulartyp wird ergänzt.	Änderung bleibt auf klar abgegrenzte Bausteine beschränkt	Review/Tests
QS-05	Nachvollziehbarkeit	Eine gespeicherte Bewertung wird automatisch bewertet.	Gesamtwert ist aus Antworten, Kriterien und Gewichtungen reproduzierbar; alle Teilwerte werden ausgewiesen	Unit-/E2E-Test

10.3 Evaluationsergebnisse

[Ergebnisse, Abweichungen und getroffene Verbesserungsmaßnahmen]

11 Risiken und technische Schulden

Risiko/Schuld	Eintritt	Auswirkung	Gegenmassnahme	Status/Owner
[Unbekannte Webtechnologie benötigt mehr Einarbeitung]	Mittel	Hoch	Früher technischer Spike; zeitlich begrenzte Evaluation	[Offen / Name]
[Lighthouse-Ziel wird spät verfehlt]	Mittel	Mittel	Regelmässige Messung mit Produktions-Build	[Offen / Name]
[Eigene technische Schuld]	[T/M/H]	[T/M/H]	[Massnahme oder bewusste Akzeptanz]	[Status / Name]

12 Glossar

Begriff	Bedeutung im Projekt
CRUD	Erstellen, Lesen, Ändern und Löschen einer Ressource
MoSCoW	Priorisierung in Must, Should, Could und Won't
[Fachbegriff]	[Kurze, eindeutige Definition]

13 Fazit und Reflexion

13.1 Was ist gut gelaufen?

[Erfolgreiche Vorgehensweisen, technische Ergebnisse und Gründe]

13.2 Wo lagen die Herausforderungen?

[Fachliche, technische und organisatorische Herausforderungen sowie der Umgang damit]

13.3 Was würde ich beim nächsten Mal anders machen?

[Konkrete Verbesserungen für Planung, Architektur, Umsetzung und Qualitätssicherung]

13.4 Zielerreichung

[Erreichte und nicht erreichte User Stories sowie Qualitätsziele; Abweichungen begründen]

14 Arbeitsjournal

Datum	Stunden	Tätigkeiten/Ergebnis	Kumuliert
[TT.MM.JJJJ]	[0.0]	[Analyse, Implementierung, Tests oder Dokumentation]	[0.0]
[TT.MM.JJJJ]	[0.0]	[Tätigkeiten und sichtbares Ergebnis]	[0.0]
Total	[0.0]	Zielwert: ca. 60 Stunden persönliche Arbeit	

15 Anhang

15.1 Abgabe und Softwareartefakte

Artefakt	Fundort
Git-Repository	[URL]
Produktive Applikation	[URL oder Startanleitung]
Testberichte	[Pfad/URL]
Lighthouse-Berichte	[Pfad/URL]
API-Dokumentation	[Pfad/URL, falls vorhanden]

15.2 Ergänzende Unterlagen

[Zusätzliche, im Haupttext referenzierte Inhalte]